

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
JNANA SANGAMA, BELAGAVI - 590018



A Project Report on

**“EDUSPHERE: AI AND WEB INTEGRATED PLATFORM FOR COGNITIVE
LEARNING”**

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Engineering

In

Department of Computer Science and Engineering

for the Academic Year: 2024-25

Submitted by

Bhumika R	1NT21C053
Swati S U	1NT21CS185
Tejashwini Hosamani	1NT21CS190
Veeksha C S	1NT21CS196

Under the Guidance of

Ms. Sowmya M. R.

Assistant Professor

Dept. of Computer Science and Engineering



YELAHANKA, BENGALURU- 560064

Department of Computer Science and Engineering

Certificate

This is to certify that the project work entitled “*Edusphere: Web and AI integrated platform for cognitive learning*” has been carried out by *Bhumika R(1NT21CS053)*, *Swati S U(1NT21CS185)*, *Tejashwini Hosamani(1NT21CS190)*, and *Veeksha C S(1NT21CS196)*, bonafide students of *Nitte Meenakshi Institute of Technology*, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in the **Department of Computer Science and Engineering**, under **Visvesvaraya Technological University**, Belagavi, during the academic year **2024–2025**.

The project report has been examined and approved as it meets the academic requirements specified under the autonomous scheme of **Nitte Meenakshi Institute of Technology** for the said degree.

Signature of the Guide

Ms. Sowmya M. R.
Assistant Professor
Nitte Meenakshi Institute of
Technology, Bengauru-560064

Signature of the HoD

Dr. S Meenakshi Sundaram
Professor and Head
Nitte Meenakshi Institute of
Technology, Bengauru-560064

Signature of the Principal

Dr. H C. Nagaraj
Principal
Nitte Meenakshi Institute of
Technology, Bengauru-560064

External Viva-Voce

Name of Examiners

Signature with Date

1.

.....

2.

.....

Acknowledgement

The successful execution of our project has been a significant milestone, and we take this opportunity to express our heartfelt gratitude to all those who have supported and guided us throughout this journey. Whatever we have achieved is the result of their encouragement and help, and we remain deeply thankful to each one of them.

We express our sincere thanks and seek the blessings of **Dr. N. R. Shetty**, Advisor, *Nitte Meenakshi Institute of Technology*, for his vision and emphasis on project-based learning and constructivist principles that have greatly enriched our academic experience. We are grateful to **Mr. Rohit Punja**, Administrator, *Nitte Education Trust*, and **Dr. Sandeep Shastri**, Vice President, *Bangalore Campus, Nitte University*, for their strategic leadership and continued support in fostering academic excellence.

We extend our special thanks to our beloved Principal, **Dr. H. C. Nagaraj**, for providing us with the necessary resources, facilities, and motivation to carry out our project successfully. Our sincere gratitude goes to **Dr. J. Sudheer Reddy**, *Dean – Academics*, and **Dr. Kiran Aithal**, *Dean – Research & Development*, for their guidance, encouragement, and for creating an environment that promotes innovation and academic growth.

We also convey our deep appreciation to **Dr. S Meenakshi Sundaram, Head of the Department**, CSE, for his constant encouragement, valuable guidance, and for fostering a vibrant learning environment. We extend our heartfelt thanks to our project guide, **Ms. Sowmya M. R.**, Assistant Professor, *Department of Computer Science and Engineering* for her unwavering support, timely feedback, and insightful mentorship throughout the project. We are also indebted to our parents for their unconditional love, support, and encouragement throughout our academic journey at *Nitte Meenakshi Institute of Technology*, Bengaluru. Finally, we would like to express our appreciation to all those—**named and unnamed**—who contributed in any way to our learning and success.

Bhumika R (1NT21CS053)

Swati S U(1NT21CS185)

Tejashwini Hosamani(1NT21CS190)

Veeksha C S (1NT21CS196)

Place: Bengaluru

Date:

Abstract

This project introduces a smart academic management platform designed to improve communication and streamline tasks for both students and teachers. The system is built with a React.js frontend and a Node.js backend, utilizing MongoDB for database management. AI integrations implemented with Python and TensorFlow enhance functionalities like attendance validation, test evaluation, and chatbot assistance. For students, the platform provides a personalized dashboard encompassing modules for attendance tracking, syllabus access, resource recommendations via web scraping, test participation with AI-driven feedback, and a dynamic calendar for managing submissions and personal events. Additionally, a chatbot and messaging system offers individual real-time assistance and communication. For teachers, the system features a dashboard for attendance management, class-wise subject tracking, notes uploads and analysis reports. AI-driven computer vision enables attendance marking through image recognition. Teachers can also share announcements directly with students and maintain a personal calendar. The platform's AI capabilities ensure accuracy in attendance proof validation and detect fraudulent documents. Furthermore, the system automates test evaluations and suggests areas for improvement, enhancing the learning process. This simplifies academic management, improves student performance, and fosters improved student-teacher collaboration.

Contents

Acknowledgement	i
Abstract	ii
List of Figures	iv
List of Table	v
CHAPTER 1 Introduction	1
1.1 Motivation	1
1.2 Organization of the Report	2
CHAPTER 2 Literature Survey, Problem Definition and Objectives	3
2.1 Background Work	3
1.1 Open Issues and Challenges	5
2.2 Problem Definition	6
2.3 Objectives	6
2.4 Scope of the Work	6
CHAPTER 3 Design Approach and Methodology	8
3.1 System Requirements	8
3.2 Architectural Design	13
3.3 Dataflow Diagram	15
3.4 Class Hierarchy Diagram	15
3.5 Use Case Diagrams	16
3.6 Sequence Diagrams	17
3.7 Activity Diagram	18
3.8 Methodology	19
CHAPTER 4 Implementation Details	30
4.1 Description of Process	30
4.2 Pseudo-code	30
CHAPTER 5 Result and Analysis	36
5.1 Testcases	36
5.2 Results	37
CHAPTER 6 Conclusion and Future Scope	43
Bibliography	44
Appendix – A	45
Appendix – B	46
Publication Details	47

List of Figures

Fig 1 Architectural Design.	13
Fig 2 Architecture of CNN model for Face Recognition.....	13
Fig 3 Architecture of CNN model for Image Editing Detection.....	14
Fig 4 Workflow of User Requests from Frontend to AI Services	15
Fig 5 Dataflow Diagram for AI Prediction.....	15
Fig 6 Class Hierarchy Diagram for an Educational Platform	15
Fig 7 Use Case Diagram: Student	16
Fig 8 Use Case Diagram: Teacher.....	16
Fig 9 Sequence Diagrams: Student.....	17
Fig 10 Sequence Diagrams: Teacher	17
Fig 11 Activity Diagram: Student.....	18
Fig 12 Activity Diagram: Teacher.....	18
Fig 13 Login page.....	37
Fig 14 Dashboard - Courses, activity, profile, top bar	38
Fig 15 Dashboard – Courses and Schedule.....	38
Fig 16 Attendance Page.....	38
Fig 17 Chatbot Interface.....	39
Fig 18 Face Recognition.....	39
Fig 19 Image Editing Classification: Real	40
Fig 20 Image Editing Classification: Fake.....	40
Fig 21 Training and Validation Accuracy and Loss graph for CNN Face Recognition.....	41
Fig 22 Training and Validation Accuracy and Loss graph for CNN Image Editing	41

List of Tables

Table 1	CNN Layer Architecture for Face Recognition	22
Table 2	CNN Layer Architecture for Image Editing Detection	25
Table 3	Comparative analysis of Face Recognition Models	36
Table 3	Comparative analysis of Face Recognition Models	41
Table 4	Comparative analysis of Image Editing Classification Models	42

CHAPTER 1 INTRODUCTION

Learning Management Systems (LMS) have become a vital part of modern education, offering an organized way to deliver and manage educational content. LMS platforms were already essential in schools and businesses, helping teachers create courses, track student progress, track attendance and grading [6], [20]. They made it easier to manage the complexities of both online and blended learning environments, simplifying many administrative tasks. Today, AI plays a significant role in the evolution of LMS. With AI, these systems can now offer personalized learning experiences, adjust content based on individual student needs, and provide real-time feedback [2], [3]. Intelligent features, like adaptive learning paths and virtual tutors, are making LMS smarter, more responsive, and better equipped to support students in their learning.

1.1 Motivation

The education sector has struggled for years with inefficiencies in resource allocation, attendance management, communication, and the lack of personalized learning experiences. Traditional methods of managing academic workflows often fall short in meeting the diverse needs of students and teachers. These inefficiencies not only consume valuable time but also hinder the overall quality of education [1], [2].

In the age of digitization, Artificial Intelligence (AI) provides promising solutions to these challenges. AI's ability to process large volumes of data, automate repetitive tasks, and provide personalized feedback can significantly enhance the efficiency and effectiveness of educational systems. By integrating AI into educational workflows, institutions can ensure streamlined processes, improved outcomes, and more engaging learning experiences [3], [5].

This project specifically aims to develop an AI-powered educational platform for both students and teachers. The platform addresses key pain points such as attendance fraud detection [9], personalized learning pathways [4], automated test creation and grading [15], [16], and efficient resource sharing [6], all of which contribute to a holistic academic environment.

AI integration is fueled by the growing need for scalable and effective educational procedures as well as the requirement for learning experiences to be personalized[2]. The

diverse needs of students and teachers are not adequately met by traditional methods, which can impact educational outcomes and overall satisfaction. AI offers a promising solution to bridge these gaps by automating repetitive tasks and improving decision-making processes [9].

1.2 Organization of the Report

This chapter provides the motivation behind the project and outlines the structure of the report.

Chapter 2: Literature Survey, Problem Definition, and Objectives

It presents a survey of existing work in the domain, highlights open issues and challenges, defines the problem, states the objectives, and outlines the scope of the project.

Chapter 3: Design Approach and Methodology

This chapter details the system design including architectural design, dataflow diagrams, class hierarchy, use case, sequence, and activity diagrams. It also explains the methodology followed during the development.

Chapter 4: Implementation Details

It describes the process followed during implementation and includes relevant pseudo-code for core modules.

Chapter 5: Result and Analysis

This chapter presents the results obtained and provides an analysis based on the outcomes.

Chapter 6: Conclusion and Future Scope

The final chapter summarizes the work done and discusses potential directions for future research and development.

CHAPTER 2 LITERATURE SURVEY

2.1 Background Work

1. AI in Education: Automated Essay Grading and Learning Systems

A significant number of papers focus on the application of artificial intelligence (AI) in education, especially for automating essay grading and supporting personalized learning [4], [6]. One notable study is the AI-based Automated Essay Grading System using NLP, which leverages deep learning models like Long Short-Term Memory (LSTM), Random Forest Regressor, and Deep Neural Networks (DNNs). Among these, the DNN model demonstrated the best performance, providing consistent and accurate essay scores [10]. Another paper, Automated Essay Scoring Using Machine Learning, explores machine learning techniques for automated essay scoring (AES), comparing models like CNNs, LSTMs, and BERT. The top-performing models, SBLSTMA (Siamese Bidirectional LSTM Neural Network Architecture) and BERT + handcrafted-features, achieved a QWK score of 0.801 [12]. In contrast, the CNN-based hierarchical CNN and SBLSTMA models achieved a score of 0.734 [10], [12]. Research on Automatic Essay Scoring of Composition Based on CNN and OR used Convolutional Neural Networks (CNNs) combined with Ordinal Regression (OR), which enhanced accuracy and efficiency in essay scoring, outperforming older methods that required manual feature extraction [10]. Additionally, the AI-based Research Companion (ARC) introduces a hybrid recommendation engine powered by GPT-4 to assist undergraduate students in research activities, enhancing the research experience by offering tailored suggestions and access to resources [15].

2. Facial Recognition and Attendance Systems

Facial recognition-based systems for automating attendance are another prominent application of AI. One paper highlights the use of Support Vector Machine (SVM), Principal Component Analysis (PCA), and Convolutional Neural Networks (CNNs) for facial recognition. The system uses libraries like OpenCV, Dlib, and PCA to detect faces accurately, improving the efficiency of attendance marking [6], [7]. Similarly, another paper employs deep learning and machine learning techniques like Haar Cascade Classifier and Local Binary Patterns Histogram (LBPH) to address challenges such as poor lighting and proxy attendance. These techniques offer robust performance, with future research aiming to surpass 90% accuracy [7]. Another study combines HOG (Histogram of Oriented Gradients), PCA, SVM, KNN algorithms, and deep learning techniques like

Eigenface/Kernel PCA, achieving an impressive 94.32% accuracy [8].

3. Deepfake Detection

The challenge of detecting deepfakes, particularly in multimedia content, is addressed in papers such as LLM-Enhanced Deepfake Detection and Learning Meta Model for Strong Generalization Deepfake Detection. The former combines Dense Convolutional Neural Networks (Dense CNN) with Large Language Models (LLMs) like InstructBLIP to detect deepfakes with high accuracy (96.5% on datasets like CelebDF) [16]. The latter uses a meta-learning approach with Dense CNNs and multimodal fusion, achieving high accuracy on datasets like FF++, DFDC, and CelebDF-V2 [16]. This approach provides clear differentiation between real and fake videos, outperforming traditional models such as Xception [16] which struggled with distinguishing real from deepfake instances.

4. Chatbots and Virtual Learning Systems

AI-driven chatbots are increasingly popular tools for supporting education and learning. Papers like Chatbot: An Automated Conversation System for the Educational Domain and Designing an Educational Chatbot for Enhanced Learning in Programming Courses highlight the use of Random Forest models for answering student queries and managing classroom interactions [13]. These systems demonstrated high performance, with the latter achieving an F-measure score of 0.870 [13]. The chatbot Ed.dy, developed during the COVID-19 pandemic, integrates organizational tools like reminders, polls, and breakout rooms to improve virtual classroom management [14]. Similarly, Programming an Educational Chatbot to Support Virtual Learning focuses on a Telegram chatbot for programming courses, which organizes student queries, provides learning resources, and tracks assignment grades. The findings suggest improved student engagement and a more efficient learning experience [14].

5. Data Scraping and Information Retrieval

Several papers focus on web scraping and data extraction to enhance the retrieval of structured and unstructured information. Data Scraping using Python for Information Retrieval on E-Commerce with Brand Keyword introduces Python-based web scraping tools like BeautifulSoup and Selenium to identify illegal listings of branded products [17]. Meanwhile, Utilizing Python for Web Scraping and Incremental Data Extraction

emphasizes Python's role in handling dynamic content and adapting to changes in website structure. It highlights challenges such as ethical concerns and the need for adaptable scraping algorithms [17]. The use of machine learning in web scraping, especially with tools like Scrapy, allows for faster and more efficient data extraction.

2.2 Open Issues and Challenges

Despite growing interest in the digitization of education, particularly through the integration of artificial intelligence (AI), several important areas remain underexplored in existing research. While much of the literature highlights the technical capabilities of AI such as intelligent tutoring systems, adaptive learning platforms, and automated grading there is a noticeable gap in studies that focus on solving the everyday, practical inefficiencies faced by educators and students.

Tasks like manual attendance tracking, subjective grading, and administrative record-keeping continue to consume a significant amount of time and effort in many institutions. AI has the potential to automate or support these functions, yet most research tends to evaluate such tools in isolation, often in controlled or pilot settings. There is limited exploration of how these technologies perform in real-world classrooms, where context, constraints, and human factors play a crucial role.

For example, while facial recognition technologies may offer a faster way to record attendance, questions around privacy, data security, and cultural acceptance remain unresolved. Similarly, AI-based grading systems show promise in reducing subjectivity, but concerns persist around fairness, transparency, and the impact on student motivation. In many cases, the discussion overlooks how these tools might alter teacher-student interactions or shift the educator's role in the learning process.

Moreover, the administrative burden on teachers is often a major challenge. Yet, there is insufficient research on how AI can be seamlessly integrated into existing educational workflows to genuinely reduce this workload without adding new layers of complexity or technical dependence.

This research aims to address that gap by not only examining what AI technologies can do, but also exploring how they can support real educational needs, enhancing teaching and learning while respecting the values and constraints of the classroom environment

2.3 Problem Definition

The current education system struggles with several inefficiencies that hinder both teaching and learning experiences. Attendance tracking often relies on outdated manual methods, leading to inaccuracies and time wastage. Students frequently lack access to personalized learning resources, limiting their ability to progress at their own pace. Communication between teachers and students remains fragmented, reducing opportunities for timely support and engagement. Furthermore, evaluation processes are typically manual, making them time-consuming, prone to error, and challenging to scale in larger institutions.

To address these issues, there is a need for a unified, AI-driven platform that can streamline educational processes. Such a platform would centralize student and teacher dashboards, automate attendance using computer vision [8], [9], provide personalized feedback [4], and enhance communication through tools like AI-powered chatbots and dynamic calendars. This solution would make education more efficient, scalable, and tailored to individual needs [11].

2.4 Objectives

- To analyze and address inefficiencies within traditional educational systems, specifically focusing on manual operations such as attendance tracking and evaluation methods [9], [10].
- To investigate the integration of AI-based solutions in education, assessing their efficiency in optimizing processes and improving learning outcomes.
- To study the impact of AI applications on student engagement and teacher productivity, in the context of automated essay grading and facial recognition technologies [8], [15].
- To review and synthesize existing research to identify both advantages and challenges of adopting AI-driven educational technologies.
- To suggest possible directions for further research and suggest strategies for developing scalable and user-centric AI innovations in education.

2.5 Scope of the Work

- Highlighting the effectiveness of hybrid models that combine deep

learning and traditional techniques.

- Identifying gaps in current systems, such as the need for better adaptability and user- friendliness.
- Emphasizing the ethical challenges and opportunities in AI-driven education, paving the way for future advancements.
- A unified dashboard offering centralized access to academic tools for students and teachers [6].
- AI-driven attendance systems capable of fraud detection using computer vision and image processing [8], [10].
- Implementation of NLP algorithms for resource recommendations [15], [16].
- Dynamic calendars and chatbots for streamlined task management and academic support.

CHAPTER 3 DESIGN APPROACH AND METHODOLOGY

3.1 System Requirements

3.1.1 Hardware Requirements

Machine	64-bit processor, 3.0 GHz Quad-Core CPU or higher
RAM	Minimum 8 GB
GPU	NVIDIA RTX 3060 or equivalent for AI workloads
Storage	SSD, 512 GB minimum
Internet	Stable connection with 230 Mbps bandwidth

3.1.2 Software Requirements

Frontend

React.js

- Version: Latest stable version
- Purpose: Building dynamic and interactive user interfaces (UI) for both student and teacher dashboards.
- Libraries/Tools: Material-UI (for responsive and modern UI components)

Backend

Node.js

- Version: Latest LTS version
- Purpose: Backend server to handle API requests, business logic, and database communication.

Database

MongoDB (NoSQL)

- Version: Latest stable version
- Purpose: Store data such as user profiles, attendance, assignments, and test results.

AI & Machine Learning

Flask (Python)

- Version: Latest stable version
- Purpose: Backend integration for AI-related features such as chatbot functionality, image recognition for attendance, and web scraping.

- Libraries/Tools:

TensorFlow.js (for AI model integration and inference in the browser) OpenCV (for computer vision-based attendance recognition) BeautifulSoup (for web scraping)

Gpt-2 LLM (for implementation of chatbot)

3.1.2.1 Functional Requirements

General Functionality:

- Secure login and authentication system.
- Responsive user interface optimized for web and mobile devices. Student

Module Requirements:

- **Dashboard:** Displays a comprehensive overview, including subjects, tests, and upcoming events [13], [14].
- Attendance Tracking: Enables students to view their attendance percentage, upload attendance proofs, and utilizes AI for fraud detection [7], [8].
- Notes Access: Allows students to download uploaded notes and access related external resources via web scraping [17].
- Dynamic Calendar: Displays test dates, assignment deadlines, and allows users to add personal events [14].
- Messaging System: Secure communication between students and teachers for academic assistance [13].
- Chatbot Support: AI-driven chatbot powered by LLMs like gpt-2 to provide real-time help [13], [14].
- Student Profile: Displays details of submitted attendance proofs, test scores, and overall progress.
- Teacher Module Requirements:
- Dashboard: Provides a class-wise summary of attendance, uploaded

resources, and test performance metrics.

- **Attendance Management:** Options to mark attendance via an Excel sheet, manual entry, or by uploading class photos for AI-based identification using computer vision. Libraries like open CV are used to preprocess the images [7], [8]. Various DL models are trained on the data and are used for the face recognition.
- **Deepfake Detection:** The attendance letters are uploaded for further updating of attendance. They are scanned to check for any editing using machine learning or deep learning models. If the image is genuine, it automatically updates the attendance without the need of human intervention[16].
- **Class Management:** Teachers can upload personal files, notes, and announcements. They can also download attendance reports, test scores, and analysis data [14].
- **Calendar Management:** Teachers can add personal or academic events for better planning.
- **Announcement System:** Teachers can create subject-specific announcements that are automatically emailed to students.

3.1.2.2Non-functional Requirements

- **Maintainability:** Modular design ensures ease of future updates and integration of new features.
- **Usability:** The interface should be intuitive and require minimal learning for users.
- **Performance:** All operations, including AI-based analysis, should execute with a latency within 5 seconds.

3.1.2.3User Requirements

Student:

REQ-1:

The system must provide a personalized dashboard displaying all academic options, such as subjects, assignments, upcoming tests, attendance percentages, and personalized alerts.

REQ-2:

The system must allow students to view attendance percentages and upload attendance proofs (e.g., medical certificates). AI must validate the uploaded proofs and detect fraudulent documents.

REQ-3:

Students must have access to the syllabus and notes uploaded by teachers. Additionally, the system must use web scraping to recommend related resources like textbooks, interview preparation material, and other learning aids.

REQ-4:

The platform must support MCQ and written tests, evaluated automatically by AI. Students must receive AI-driven feedback with topic-specific revision suggestions.

REQ-5:

A dynamic calendar must display all upcoming assignments, submissions, and tests. Students should also have the ability to add personal events like presentations or extracurricular deadlines.

REQ-6:

A chatbot powered by LLMs must provide real-time assistance, addressing platform navigation and academic queries.

REQ-7:

The platform must include a secure messaging system enabling communication between students and teachers.

REQ-8:

Students must have access to a centralized assignment and test board displaying all pending and completed assignments and exams.

REQ-9:

The system must feature a student profile section where students can view submitted details, test scores, and attendance proof history.

Teacher User Requirements

REQ-1:

The system must provide a dashboard tailored for teachers, summarizing attendance, uploaded resources, test results, and class performance for each subject.

REQ-2:

Teachers must have multiple attendance management options, including:

- Manual marking.
- Class photo upload for AI-based attendance recognition using computer vision.

REQ-3:

Teachers must have access to a personalized calendar to manage academic events and set reminders for personal tasks like presentations and deadlines.

REQ-4:

The system must allow teachers to upload personal files such as lecture notes, assignments, and test materials.

REQ-5:

Teachers must manage class-wise subjects, including attendance tracking, note uploads, and performance tracking.

REQ-6:

Teachers must be able to upload test marks, with analysis report generation.

REQ-7:

The system must allow teachers to post class-specific announcements, which are emailed directly to all registered students for that subject.

REQ-8:

Teachers must be able to analyze student performance and download class performance reports for detailed insights into individual and group progress.

REQ-9:

AI must facilitate attendance marking using computer vision, ensuring accuracy in identifying students from uploaded class photos.

3.2 Architectural Design

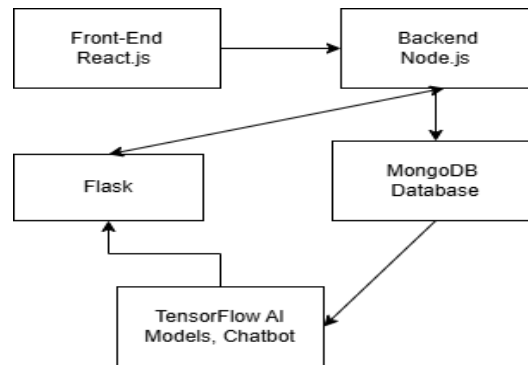


Fig. 1: Architectural Design

Fig. 1 depicts an architectural diagram of the LMS. It depicts the key components involved, including a Front-End React.js, a Backend Node.js, a Flask framework, a MongoDB database, and TensorFlow AI models and a Chatbot.

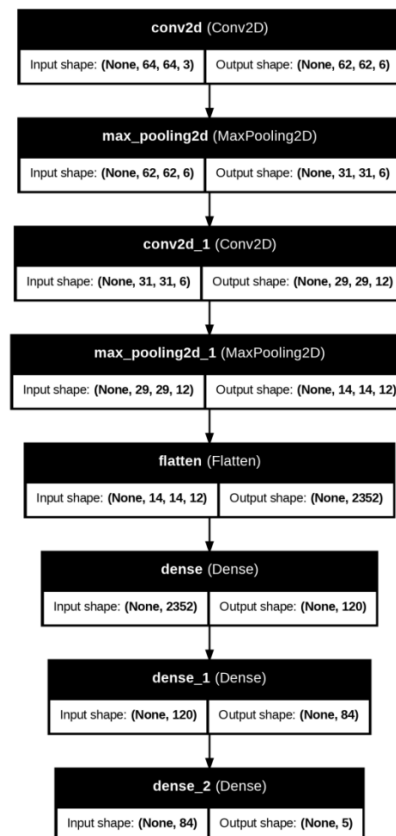


Fig. 2: Architecture of CNN model for Face Recognition

Fig. 2 depicts the model architecture that has of two convolutional layers followed by max- pooling layers, which extracts features from input images, followed by a

flattening layer that reshapes the data, and then three fully connected dense layers that enable the model to learn complex patterns. The output layer has 5 units, for the 5 classes in the classification task.

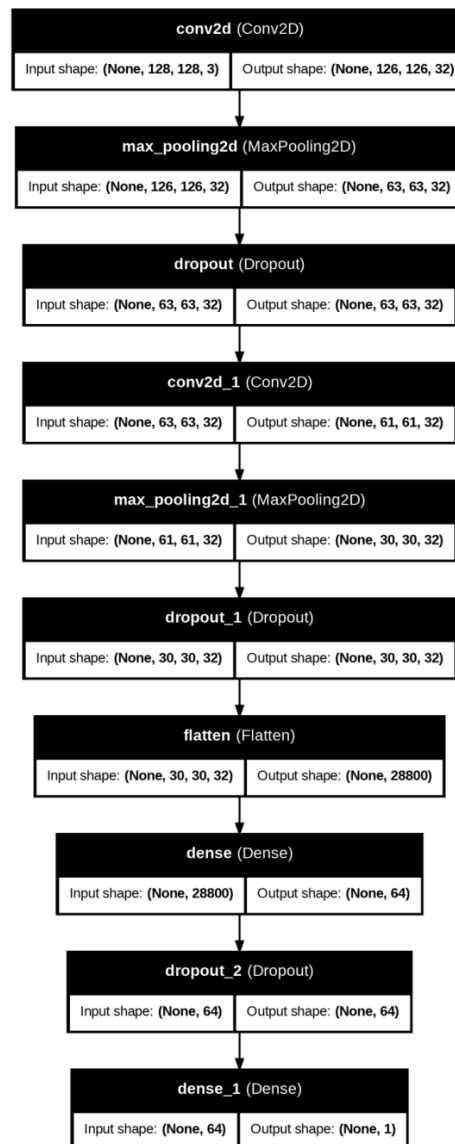


Fig. 3: Architecture of CNN model for Image Editing Detection

The model architecture in fig. 3 has two convolutional layers with max-pooling and dropout layers. The data is then flattened and passed through two fully connected dense layers, with dropout applied to prevent overfitting. The output layer has a single neuron for binary classification, predicting one of two classes.

3.3 Dataflow Diagram

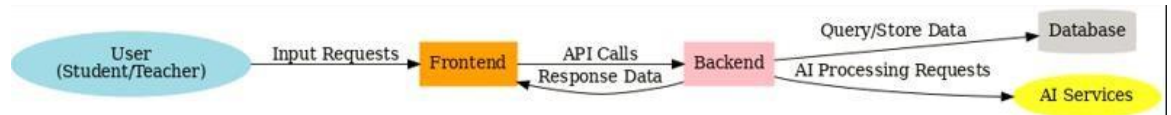


Fig. 4: Workflow of User Requests from Frontend to AI Services

Fig. 4 depicts the dataflow diagram of a web application system. It shows the sequence of flow of data and requests between different components of the application.

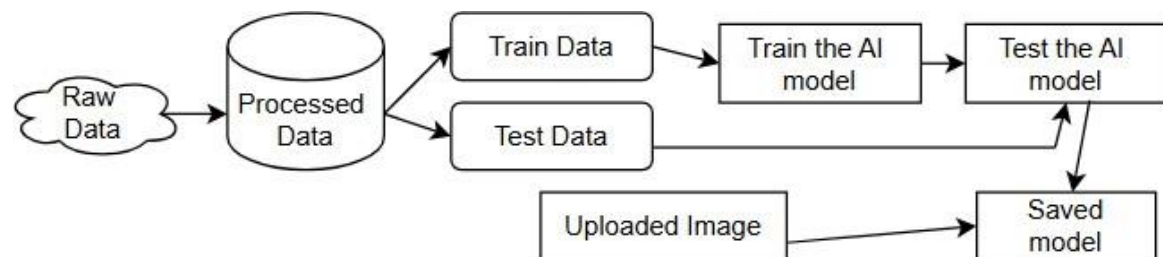


Fig. 5: Dataflow Diagram for AI Detection

Fig. 5 depicts the workflow for training and testing an AI model. It shows the steps involved, including processing raw data, training the AI model, testing the AI model, and saving the trained model. The uploaded image is later used for testing the AI model.

3.4 Class Hierarchy Diagram

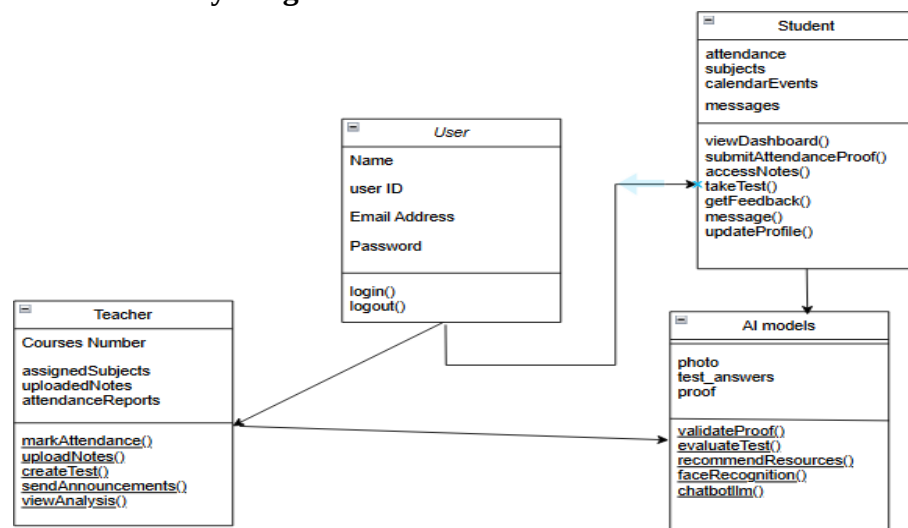


Fig. 6: Class Hierarchy Diagram for an Educational Platform

Fig. 6 depicts a class hierarchy diagram for an educational platform. It shows the key entities involved, including Students, Teachers, and AI models, along with the interactions and functionalities between them, such as attendance tracking, test taking, and AI-powered features like proof validation and face

recognition.

3.5 Use Case Diagrams

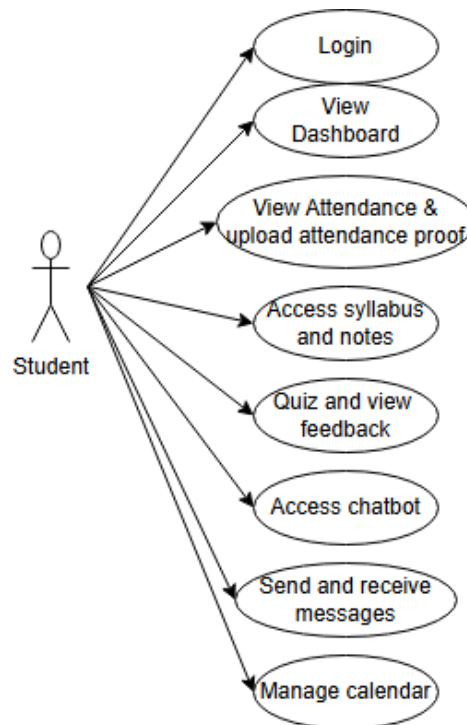


Fig. 7: Use Case Diagram: Student

Fig. 7 depicts the student use case diagram. It shows the various functionalities and features available to the student, including logging in, accessing the dashboard, viewing attendance and uploading proof, accessing course materials, interacting with a chatbot, sending and receiving messages, and managing their calendar.

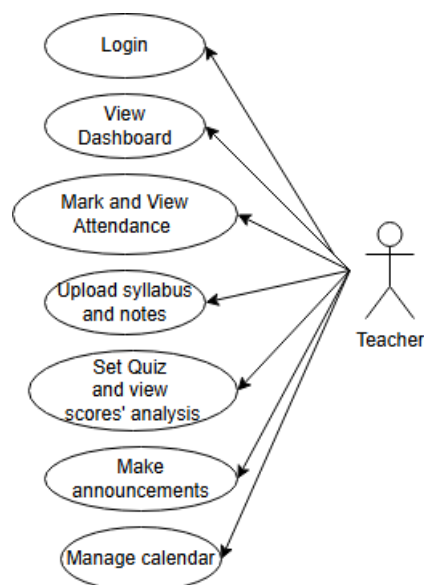


Fig. 8: Use Case Diagram: Teacher

Fig. 8 depicts a use case diagram for a teacher, which shows the functionalities and features available like logging in, accessing the dashboard, marking and viewing attendance, uploading syllabus and notes, viewing scores' analysis, making announcements, and managing the calendar.

3.6 Sequence Diagrams

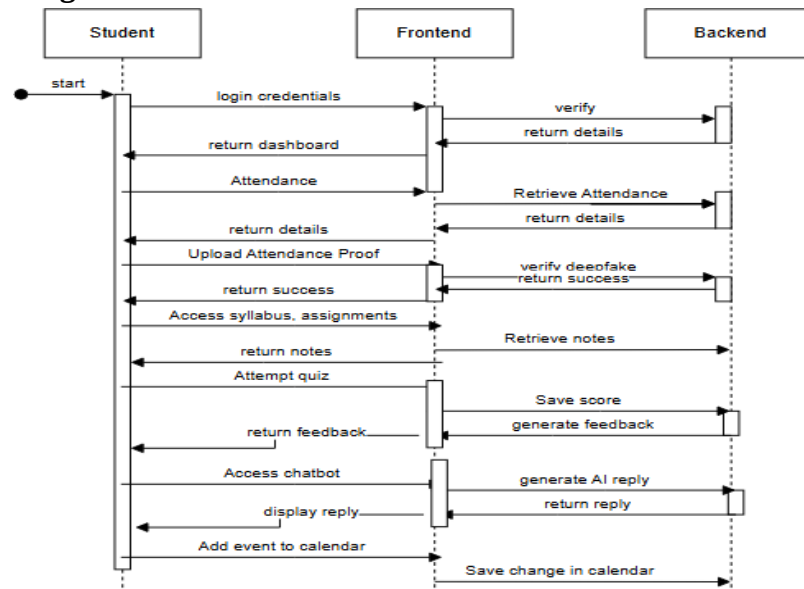


Fig. 9: Sequence Diagram: Student

Fig. 9 depicts a user sequence diagram for a student interacting with a frontend and backend system in an educational platform. It shows the various functionalities and features available to the student and the detailed data flow from the student's input to the frontend, its transfer to the backend and the corresponding response.

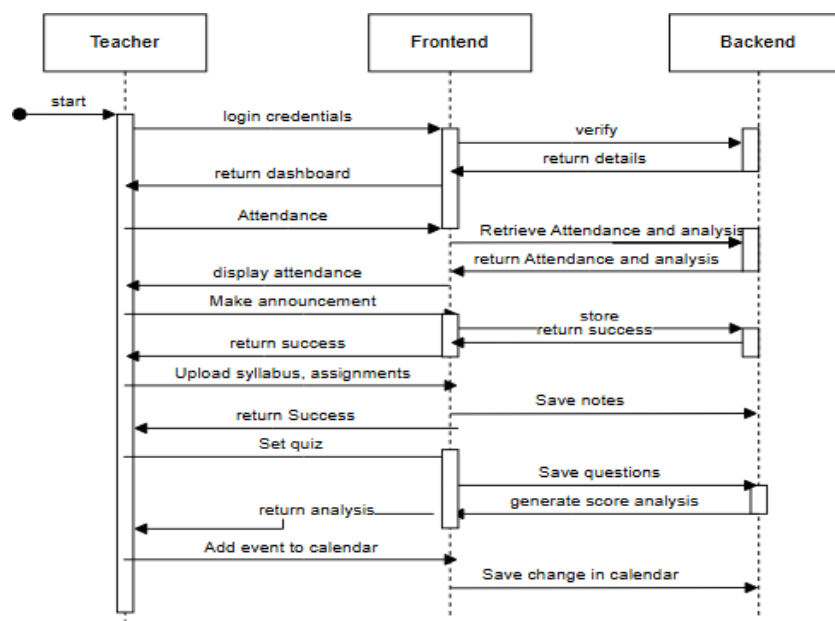


Fig. 10: Sequence Diagram: Teacher

Fig. 10 depicts a user interface flow diagram for a teacher interacting with a frontend and backend system in an educational platform. It shows the various functionalities and features available to the teacher and the detailed data flow from the teacher's input to the frontend, its transfer to the backend and the corresponding response.

3.7 Activity Diagram

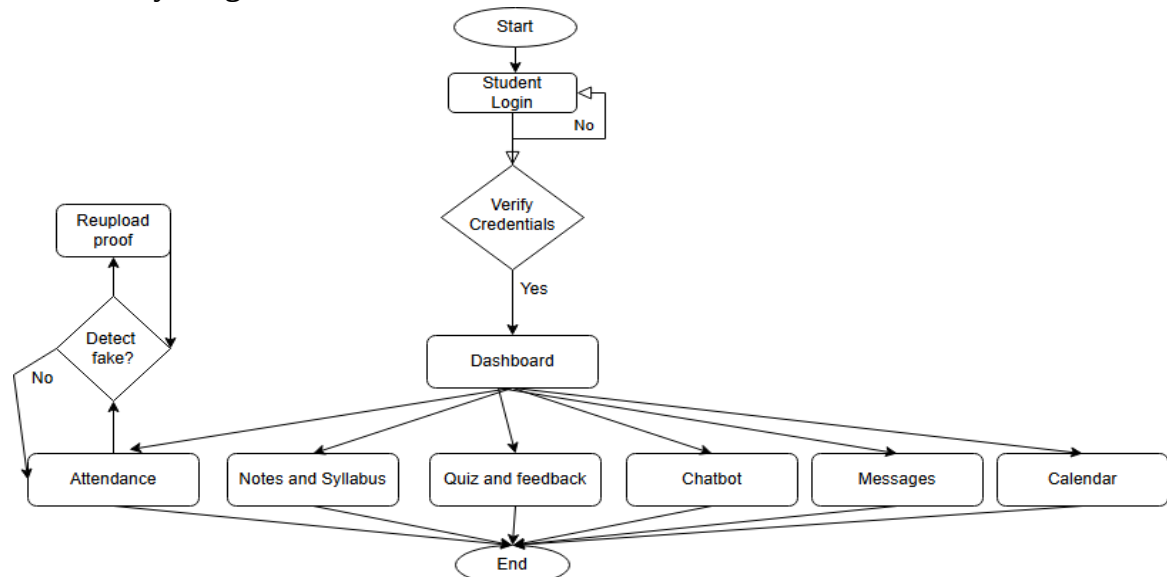


Fig. 11: Activity Diagram: Student

Fig. 11 depicts the flowchart for a student interacting with an educational platform. It shows the choices a user can make and the different conditions and their results.

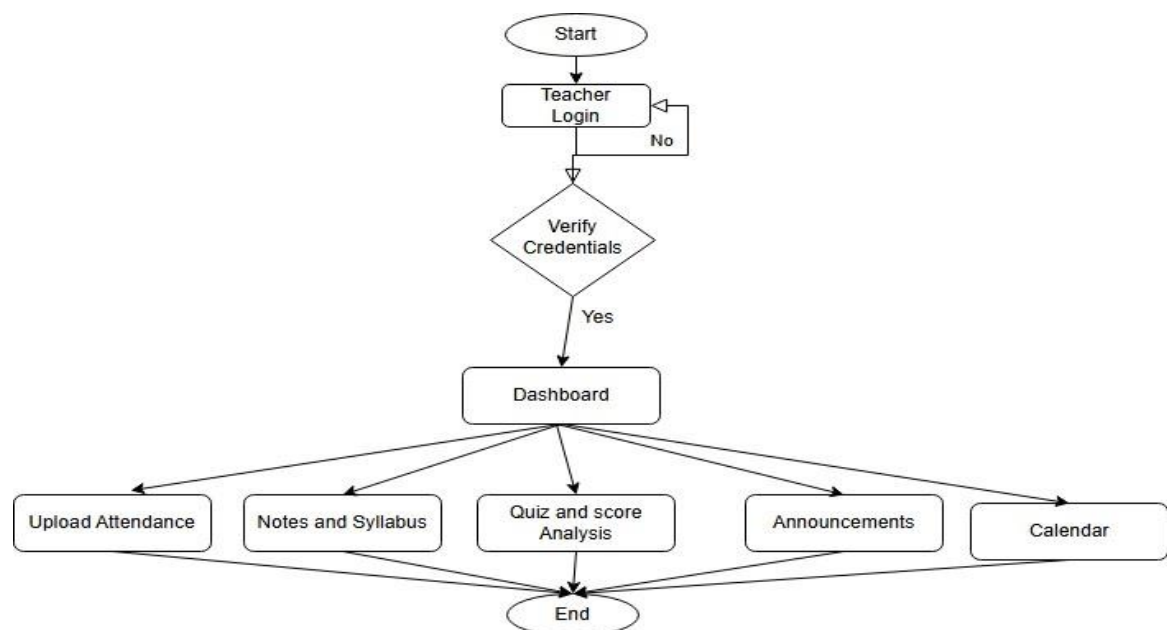


Fig. 12: Activity Diagram: Teacher

Fig. 12 depicts the flowchart for a student interacting with an educational platform. It shows the choices a user can make and the different conditions and their results.

3.8 Methodology

Given below is the proposed system.

1.Requirements Analysis and Planning

Identify Stakeholders: Students, teachers, and administrators.

Technology Stack:

Frontend: ReactJS Backend: Node.js, Flask

Database: MongoDB for data storage.

AI/ML Libraries: TensorFlow, OpenCV

Web Scraping Tools: BeautifulSoup

Cloud Services: AWS or Azure for deployment and storage

2. Chatbot

Procedure:

The chatbot interacts with two separate models from Hugging Face, a platform for Natural Language Processing (NLP) and other AI models [13], [15]. Using the requests library in Python, the chatbot sends input to the Hugging Face API by making an HTTP POST request. Each model has a unique endpoint, specified by the API URL [14]. The Hugging Face models process the input. For the BART model, it performs text summarization, while the BERT model processes a question and provides an answer from the provided context [6], [13]. The response from the Hugging Face API is then returned as a JSON object, which contains the output from the model and is displayed to the user [13].

Features:

- Text Summarization: The BART model (facebook/bart-large-cnn) is used for summarizing long pieces of text into a concise and meaningful summary [6], [15]. It leverages a combination of transformer-based architectures to generate coherent summaries.
- Question-Answering: The BERT model (google-bert/bert-large-uncased-whole-word-masking-finetuned-squad) is used for question answering, where the chatbot receives a

question and a related context. The model then identifies and extracts the correct answer based on the context provided [13], [15].

Tools and Libraries:

Hugging Face API: Hugging Face provides pre-trained transformer models that can be used for NLP tasks like question-answering, text summarization, and text generation. It simplifies the use of state-of-the-art NLP models by exposing them through an API.

Requests Library: The Python requests library is used for making HTTP requests. It sends POST requests with a JSON payload to interact with the Hugging Face API, allowing the chatbot to retrieve results from the models.

JSON: The response from the Hugging Face API is returned in JSON format. This data is then parsed and extracted in the Python code to display the output.

Accuracy and Performance:

High accuracy and context relevance were achieved due to usage of LLMs from HuggingFace. The performance of both models is reliant on the underlying infrastructure of Hugging Face's servers. The system's response time can be influenced by factors like the size of the input, complexity of the question.

3. Attendance Management System: Face Recognition

Dataset:

The dataset consists of a root folder containing 5 subfolders. Each subfolder is named after a person (e.g., "angelina_jolie", "bradley_cooper", etc.) and contains 1 image of that person. Each subfolder represents a unique class (a person), and the corresponding image serves as the training image for that class. The dataset is used for image recognition, where the objective is to predict which person is in the image based on features extracted from the images [7], [8].

Preprocessing:

- Background Removal (using GrabCut):** The code uses the GrabCut algorithm to remove the background of each image. This isolates the foreground (i.e., the person's face),

making the face detection and feature extraction steps more efficient. The background is removed using a mask and a defined rectangle, and only the foreground (person) is retained for processing.

- Face Detection:** The Haar Cascade classifier is used for detecting faces within the images. This ensures that only the face portion of the image is cropped and used for training, improving the model's accuracy [8].

- Normalization:** After preprocessing, images are normalized using a MinMaxScaler. This scales the pixel values to a range of [0, 1], which helps improve the performance of ML models by standardizing the input data [7].

- HOG Feature Extraction:** Histogram of Oriented Gradients (HOG) is computed for each image. HOG captures the shape and appearance of objects by analyzing gradients and orientations in the image. It's a frequently used parameter for object and face recognition [8].

- Augmentation:** Data augmentation is applied to increase the number and variety of training images, which is crucial for DL models. Techniques such as rotation, zoom, shear, and horizontal flipping are used to artificially create more images from the existing ones [7].

Model Training:

The code compares three machine learning models:

- Support Vector Machine (SVM):**

Feature Extraction: The SVM model uses HOG features as input to classify the images. The HOG features are used for capturing spatial and textural information about faces.

Training: The SVM classifier was trained with a linear kernel and probability estimates enabled.

- **K-Nearest Neighbors (KNN):**

Feature Extraction: Like SVM, the KNN model also uses HOG features. KNN works by classifying a new image based on the majority label of the nearest neighbors (images with similar features).

Training: The KNN model was trained using Euclidean distance as the metric for similarity and uses 5 nearest neighbors.

- Convolutional Neural Network (CNN):

Architecture: The CNN used consists of two convolutional layers, each followed by max-pooling layers, followed by a fully connected layer and a softmax output layer.

Training: The model is trained using categorical cross-entropy loss and Adam optimizer. The training is done for 2 epochs, which is relatively low, likely due to the small size of the dataset (5 people with just 1 image per person). The images are resized to 64x64 pixels to match the input size required by the model.

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 62, 62, 6)	168
max_pooling2d (MaxPooling2D)	(None, 31, 31, 6)	0
conv2d_1 (Conv2D)	(None, 29, 29, 12)	660
max_pooling2d_1 (MaxPooling2D)	(None, 14, 14, 12)	0
flatten (Flatten)	(None, 2352)	0
dense (Dense)	(None, 120)	282,360
dense_1 (Dense)	(None, 84)	10,164
dense_2 (Dense)	(None, 5)	425

Table 1: CNN layer architecture for face recognition

Accuracy and Performance:

- CNN Model:

The CNN model achieved an accuracy of 99.37% on the test set. This indicates that the model performs well but has a slight chance of misclassification. The high performance of CNN is due to the ability to learn hierarchical features from the image data [8].

- SVM Model:

The SVM classifier achieved 100% accuracy on the test set. This suggests that the SVM model was able to perfectly classify the images based on the HOG features [7]. The

confusion matrix demonstrates that there were no misclassifications, and each class (person) was correctly identified.

- KNN Model:

The KNN classifier also achieved 100% accuracy on the test set. The classification report and confusion matrix show perfect performance, with all test samples being classified correctly.

Tools and Libraries:

- OpenCV (cv2): OpenCV is used extensively for image processing tasks, such as reading images, face detection using Haar cascades, background removal using GrabCut, and image resizing. It provides a rich set of functions for computer vision tasks.

- NumPy: NumPy is used for handling arrays and matrices, especially for image processing and manipulation. It helps in creating masks for background removal and manipulating image arrays for normalization and feature extraction.

- Matplotlib: Matplotlib is used to visualize images during the preprocessing and augmentation steps. It's also used for displaying the classification results, such as confusion matrices and images.

- Scikit-learn: Scikit-learn is used for feature extraction (HOG), model training, and evaluation. It provides easy-to-use functions for ML tasks like training classifiers (SVM, KNN) and evaluating performance of model with metrics like precision, accuracy, recall, and F1-score.

- Keras: Keras, part of TensorFlow, is used to build and train the Convolutional Neural Network (CNN). It provides a high-level interface for deep learning, allowing for easy construction of neural network layers and training models with backpropagation.

- scikit-image: The HOG feature extraction is done using scikit-image, a library for image processing in Python. It provides a straightforward way to compute HOG features, which are crucial for the SVM and KNN classifiers.

4. Attendance Management System: Image Editing Detection

Dataset:

The dataset used in the code is the CASIA2 dataset, which is available on Kaggle. The dataset is structured into two subfolders. Au (Authentic) folder contains authentic (real) images. Tp (Tampered) folder contains tampered (fake) images. Each subfolder has multiple image files, typically in formats such as .jpg, .png. The images are used for deepfake detection, where the model differentiates between real and tampered images.

Preprocessing:

- **Balancing the Dataset:** To ensure that the model is not biased towards one class, the dataset is balanced. The number of images in the Au (Authentic) folder is undersampled to match a specified target size (target_size) [16]. All images from the Tp (Tampered) folder are retained, ensuring both categories are present in equal proportions during training. The preprocessed images are saved into a new folder structure where the Au and Tp folders contain a balanced number of images.
- **ELA:** Error-level analysis is used to detect image tampering by saving the image with reduced quality and comparing the original image with the recompressed version [16]. Differences in error levels across the image help highlight tampered regions.
- **Edge Detection:** Canny edge detection is used to highlight the boundaries and features within an image, which can be used for identifying signs of tampering [7]. It facilitates the extraction of meaningful features from the image that can be applied to model training.
- **Image Resizing:** Images are resized to a consistent size of 128x128 pixels to standardize the input for DL models. This makes sure that all images are in the same format, and the models can process them efficiently.

3.6.1 Model Training:

The code compares three machine learning models:

- **Hybrid CNN+LSTM (Long Short-Term Memory):**

Architecture: This hybrid model combines CNN and LSTM layers. The LSTM layers are utilized to record sequential dependencies, which is helpful for combining both spatial and temporal variables, while the CNN layers extract spatial features from the images. [16].

Training: Similar to the CNN model, the CNN+LSTM model uses binary cross-entropy loss and the Adam optimizer. The LSTM part is intended to capture more complex

temporal relationships between features learned from the CNN layers, enhancing the model's performance in distinguishing tampered images.

- Random Forest Classifier:

Architecture: A traditional machine learning model, the Random Forest Classifier uses a collection of decision trees to make predictions. It is trained on features extracted from the images, such as Error-Level Analysis (ELA) and edge features.

Training: The Random Forest model is trained on the features extracted (ELA and edge intensity) rather than raw pixel values. Random Forests are used for classification tasks, and here, it is used to differentiate between authentic and tampered images based on hand-crafted features.

- Convolutional Neural Network (CNN):

Architecture: The CNN model uses convolutional layers followed by max-pooling layers. These layers are designed to automatically learn spatial hierarchies of features (edges, textures, shapes, etc.) from the input images.

Training: It was trained using binary cross-entropy loss and the Adam optimizer. The training lasted for 14 epochs. The CNN is trained to classify images into two classes: authentic or tampered.

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
dropout (Dropout)	(None, 63, 63, 32)	0
conv2d_1 (Conv2D)	(None, 61, 61, 32)	9,248
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 32)	0
dropout_1 (Dropout)	(None, 30, 30, 32)	0
flatten (Flatten)	(None, 28800)	0
dense (Dense)	(None, 64)	1,843,264
dropout_2 (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65

Table 2: CNN layer architecture for Image Editing Detection

Accuracy and Performance:

- CNN:

Test Accuracy: 89.35%

Test Loss: 0.3168 The CNN model achieved a high accuracy in classifying images into authentic and tampered categories. The relatively high accuracy indicates that CNNs are effective at learning complex features from images.

- Hybrid CNN+LSTM: Test Accuracy: 89.12%

Test Loss: 0.2385 The hybrid CNN+LSTM model achieved similar accuracy to the CNN model. The LSTM layers may provide an advantage in capturing temporal patterns in image sequences, although it showed a marginally lower accuracy than the CNN model.

- Random Forest:

Test Accuracy: 77.25%

Classification Report: Precision (Authentic): 0.80, Recall (Authentic): 0.74, F1-Score (Authentic): 0.77. Precision (Tampered): 0.74, Recall (Tampered): 0.80, F1-Score (Tampered): 0.77 While the Random Forest model achieved lower accuracy compared to the CNN and CNN+LSTM models, it still performed reasonably well. The model achieved balanced precision and recall for both classes, with an average F1-score of 0.77.

Tools and Libraries:

- OpenCV (cv2): Used for image manipulation, resizing, edge detection, and error-level analysis (ELA).

- PIL (Pillow): Used for image opening, saving, and image enhancement during ELA.

- Joblib: Used for saving the trained Random Forest model to disk for future use.

- TensorFlow and Keras: Used for building and training DL models, like CNN and CNN+LSTM architectures.

5.Web Scrapping

Procedure:

The user is prompted to input a topic they wish to search for. This is achieved using the `input()` function, where the input is stored in the `topic` variable. The user is prompted to input a topic they wish to search for. The inputted topic may contain spaces or special characters. To handle these, the topic is encoded using `urllib.parse.quote_plus()`, which replaces spaces with `+` and encodes any special characters for safe inclusion in a URL [17]. The function `search_internet` constructs the URL for the Google Custom Search API. The base URL for the API is `https://www.googleapis.com/customsearch/v1`, and query parameters are appended. A GET request is made to the Google Custom Search API using the `requests.get()` method. The query parameters are passed along with the request.

Features:

•Topic Search:

The core feature of the program is the ability to search the internet for a user-defined topic by querying the Google Custom Search Engine.

•Display Search Results:

After querying the API, the top 10 search results are retrieved and displayed. For each result, the details shown are Title which is the title of the search result, URL which is a link to the webpage containing the result, snippet which is a brief description or snippet of the content available at the result's URL..

Tools and Libraries:

•Beautiful Soup: For parsing HTML content.

•Requests library: The requests library is used to make HTTP requests to the Google Custom Search API. It is simple to use and handles various HTTP methods (GET, POST, etc.). In the code, the `requests.get()` function sends a GET request to the Google Custom Search API, passing query parameters and receiving the response.

•urllib.parse Library: The `urllib.parse` module is used to handle URL encoding and decoding. Specifically, the `urllib.parse.quote_plus()` function encodes the user input (search topic) into a format that can be safely included in a URL, replacing spaces with `+` and ensuring that special characters are properly encoded.

- Google Custom Search API: The Google Custom Search API is a web service that allows developers to retrieve web pages from Google's search engine based on a custom search engine ID. In the code, the API is accessed to search the web for the given topic and return a list of results that match the query.

Accuracy and Performance:

The scraping system demonstrated high relevance in extracting topic-specific content, achieving significant precision in filtering out unrelated data. The speed of the search operation is largely determined by the performance of the Google Custom Search API, which depends on factors like the network speed and the load on Google's servers. However, since the program retrieves only 10 results, the response time is fast.

6.Dynamic Calendar and Task Management

Frontend Development:

Use React components like modals and calendars [7], [13]. Integrate notifications using libraries like socket.io-client for real-time updates.

Backend Integration:

Create APIs to fetch events and tasks. Automate reminders for deadlines using cron jobs or third- party libraries like Agenda.js.

LMS Frontend Development

User Interface:

Implement React components for dashboards, course catalogs, and player interfaces. Style using Material-UI or custom CSS frameworks.

State Management: Use Context API or Redux for managing global state across components.

APIs and Data Binding:

Fetch data dynamically from the backend using axios or fetch. Display progress and analytics with tools like Chart.js.

User Authentication:

Use Firebase or custom backend logic for secure login/signup flows. Implement role-based access controls [15].

Backend Development

API Development:

Use Express.js to create RESTful APIs. Implement routes for user authentication, resource fetching, and task management.

Data Handling:

Use MongoDB for storing and retrieving user, attendance, chatbot, and course data. Optimize database queries for faster performance [13], [15].

5. Deployment and Maintenance

Deployment:

Host the frontend on platforms like Netlify or AWS Amplify. Deploy the backend and database on AWS EC2, Heroku, or Azure. Use CI/CD pipelines for smooth deployment and updates.

Monitoring:

Set up logging and monitoring using tools like Logstash or AWS CloudWatch. Address user feedback and maintain system performance.

CHAPTER 4 IMPLEMENTATION DETAILS

4.1 Description of Process

1. Chatbot Development

A chatbot was created using OpenAI's pre-trained language model, which could understand and respond to user queries. It was integrated with a backend system (e.g., Flask or FastAPI) to handle user inputs and provide answers. The chatbot could assist with educational tasks like sharing assignment details or answering questions. A user-friendly interface was provided for easy access, and the chatbot was tested to ensure its responses were accurate.

2. Attendance Management System

A face recognition system was designed using Convolutional Neural Networks (CNNs). Facial images were used to train the model to recognize faces accurately. The system detected and identified faces from images and marked attendance automatically. OpenCV helped in capturing faces, and the system was optimized to work under different lighting and angles.

3. Web Scraping Module

The BeautifulSoup library was used to gather educational resources from websites. Only links and materials related to specific topics were kept, and unrelated content was removed. The collected information was organized and displayed neatly for users to access. Scripts were tested to handle different website structures and ensure they worked even if pages changed.

4. Frontend and Backend

The user interface was made using ReactJS to make it easy to use as well as visually appealing. The backend, built with Node.js, managed data like attendance records, chatbot messages, and web scraping results. APIs connected the frontend and backend to ensure smooth communication.

4.2 Pseudo-code

1. Computer Vision and Deepfake Detection

Attendance System:

Edusphere:

```
START

LOAD dataset of images

PREPROCESS images (augment, normalize, resize)

EXTRACT features using HOG and masking

CLASSIFY faces using CNN

IF match found THEN

    MARK attendance

ELSE

    FLAG for manual verification

END

LOAD classroom dataset

SAVE trained model

END
```

Deepfake Detection:

```
START

LOAD deepfake datasets (e.g., image manipulation) PREPROCESS images
(extract and enhance features, resize) TRAIN CNN model with visual data

TEST on images

RETURN accuracy metrics and saved model END
```

2. Chatbot

Data Preparation:

```
START

LOAD educational datasets

PREPROCESS text (cleaning, tokenization, TF-IDF) EXTRACT features
(question types)

SAVE structured dataset END
```

Chatbot Functionality:

START

INPUT user query

GENERATE response using LLM models END

RETURN response END

3.Web Scraping

START

DEFINE URLs to scrape

SETUP libraries (BeautifulSoup)

FOR each URL:

 FETCH web page content

 PARSE and EXTRACT required data

SAVE data to database

IF website structure changes THEN

 ADAPT scraping script

END

RETURN extracted data

END

4. Frontend

Authentication and Authorization:

START

CREATE React forms for Login, Signup

VALIDATE inputs using react-hook-form

Edusphere:

CALL authentication API (e.g., Firebase)

CHECK role-based access

IF authorized THEN

 ALLOW access

ELSE

 DENY access

END

Dashboard:

START

FETCH user data (courses, progress)

DISPLAY data dynamically using useEffect

SHOW visual analytics with chart.js

END

Course Catalog and Enrollment:

START

FETCH course list from backend

DISPLAY courses using reusable components

APPLY filters and sorting for search

IF user clicks 'enroll' THEN

 SEND enrollment request to backend

END

Course Player:

START

LOAD course content (videos, PDFs)

IF video THEN

 PLAY using react-player

Edusphere:

IF document THEN

VIEW using react-pdf

IF quiz THEN

EMBED quiz

END

Progress Tracking and Notifications:

START

FETCH user progress data

DISPLAY progress using progress bars

SETUP WebSocket for real-time notifications

IF notification received THEN

DISPLAY notification

END

END

5. Backend with Node.js

START

SETUP Node.js server

CONFIGURE routes for API endpoints (auth, courses, notifications)

CONNECT database (e.g., MongoDB, SQLite)

HANDLE requests:

IF POST THEN

SAVE data to database

IF GET THEN

FETCH data from database

RETURN response to frontend

END

Edusphere:

Task Scheduling:

Pseudo

Copy code

START

CREATE dynamic calendar

SETUP task reminders using cron jobs

SEND notifications for upcoming deadlines

END

CHAPTER 5 RESULT AND ANALYSIS

5.1 Testcases

Test ID	Case(TC)	Description	Test Step	Expected Result	Status
TC1		Ensure all links are working properly.	Click on various links on the page	Links take user to another webpage	Pass
TC2		Login	Successful login	User is redirected to the respective dashboard	Pass
TC3		Accessing Syllabus and Notes	Navigate to "Syllabus/Notes" and select a subject.	Displays syllabus and notes uploaded by the teacher; supplementary resources are provided through web scraping.	Pass
TC4		Chatbot Functionality	Open chatbot and ask a query.	Chatbot responds with the appropriate explanation.	Pass
TC5		Dynamic Calendar Functionality	Displays all relevant dates and allows adding personal events.	Displays all relevant dates and allows adding personal events.	Pass
TC6		AI Attendance Verification	Teacher uploads a class photo for attendance.	AI recognizes students and marks attendance accurately.	Pass
TC7		Web Scraping for Resources	Search for supplementary material for a specific subject.	Relevant resources and links are fetched and displayed using web scraping.	Pass

Table 3: Test Cases and their Results

5.2 Results

The evaluation of machine learning models relies on several key metrics to assess their performance. Accuracy provides an overall measure of how many predictions were correct. Precision and Recall offer a deeper understanding of the model's ability to identify positive cases, with precision focusing on the correctness of positive predictions and recall measuring how many actual positives were captured. The F1-score combines precision and recall into a single metric, balancing the trade-off between the two. Additionally, Test Accuracy indicates how well the model generalizes to unseen data, while Test Loss measures the model's error, with lower values suggesting better performance. Together, these metrics give an insight into a model's strengths and weaknesses.

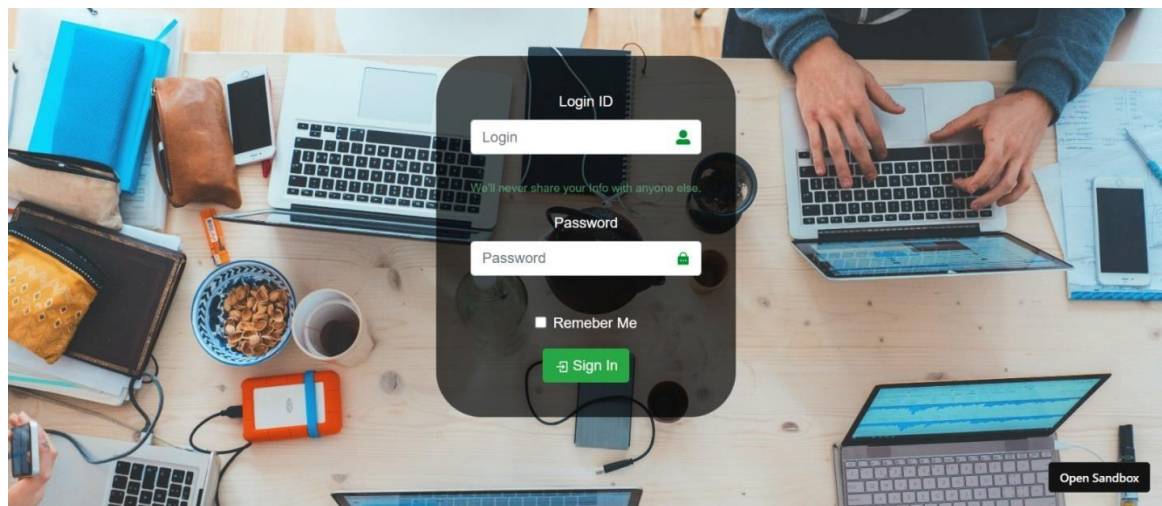


Fig. 13 Login page

Fig. 13 shows the login screen with fields for entering a login ID and password. The entered details are authenticated by the backend and then access is allowed based on whether the details match. or not.

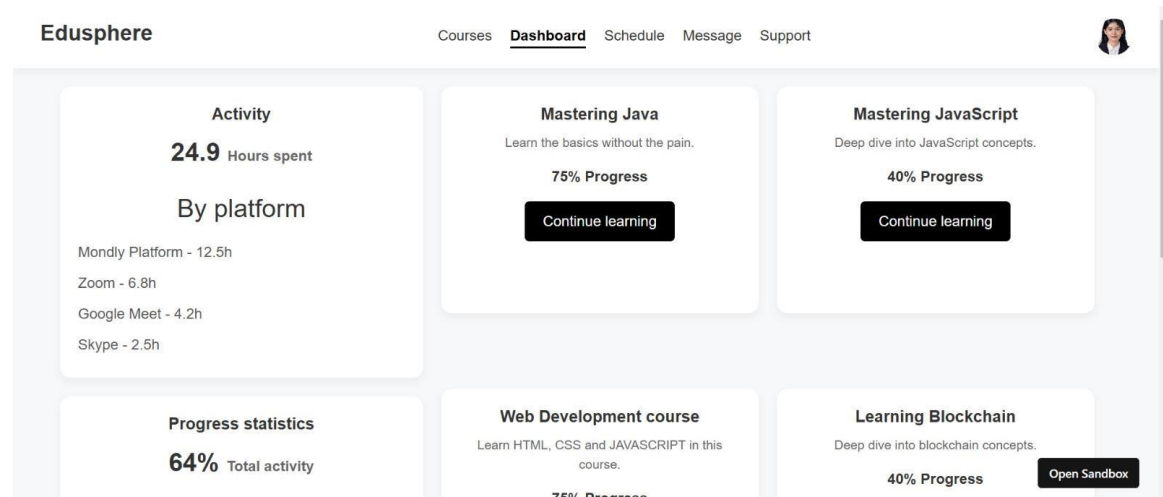


Fig. 14 Dashboard - Courses, activity, profile, top bar

Fig. 14 shows the dashboard of the LMS Edusphere. It displays the user's activity, including the time spent on different platforms, as well as their progress in various courses.

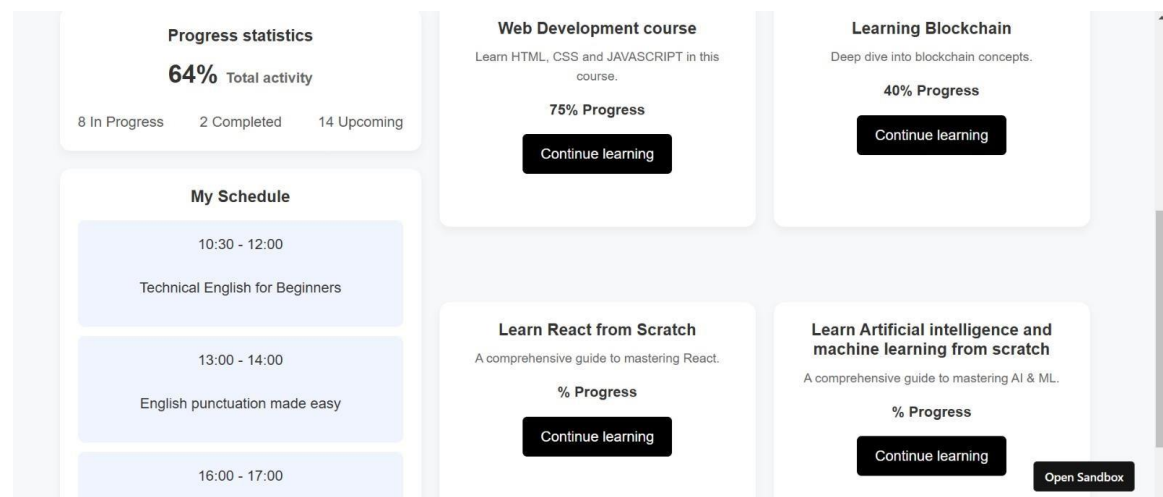


Fig. 15 Dashboard- Schedule and courses

Fig. 15 shows the user's progress statistics, their schedule, and various courses the students have registered for.

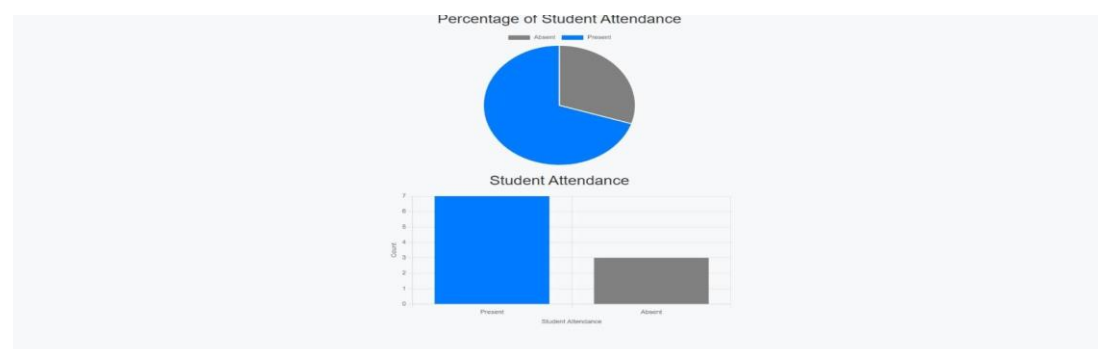


Fig. 16 Attendance Page

Fig. 16 shows a pie chart and bar chart representing the percentage of attendance. The chart is divided into Present and Absent section, indicating the proportion of students who were present and absent, respectively.

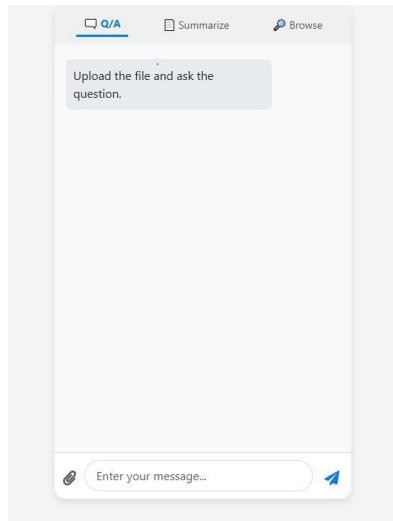
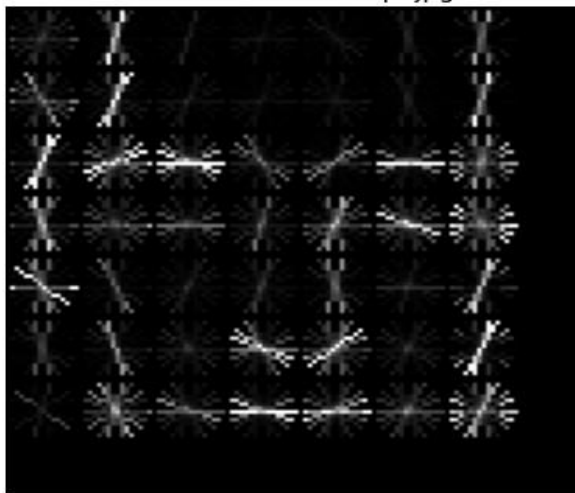


Fig. 17 Chatbot Interface

Fig. 17 shows a chat interface for a chatbot. The user can enter their message in the input field, either a question answer or summarization of a specific paragraph of a topic or to browse about a specific topic and click the Send button to communicate with the chatbot.

Preprocessed input image shape: (1, 64, 64, 3)

HOG Visualization: pr.jpg



1/1 — 0s 75ms/step

Predictions: `[[0.000000e+00 3.1351868e-15 0.000000e+00 1.000000e+00 0.000000e+00]]`

Predicted person: paul_rudd

Fig. 18 Face Recognition

Fig. 18 shows the face recognition model's results, the preprocessed input image and the predicted output.

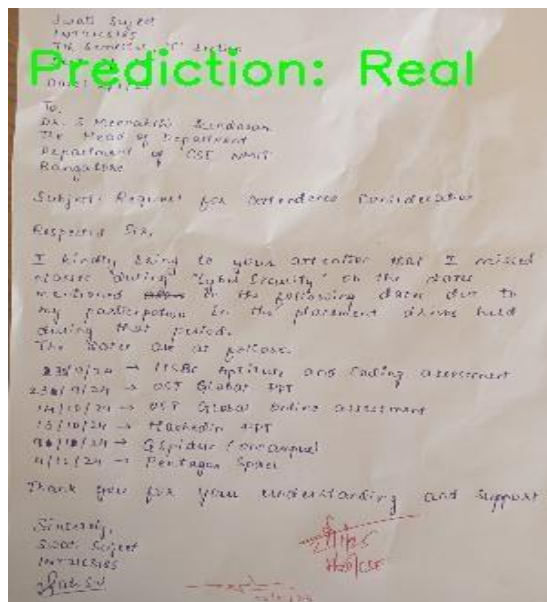


Fig. 19 Image Editing Classification: Real

Fig. 19 shows the image editing detection model's results, the input image and the predicted output as real.

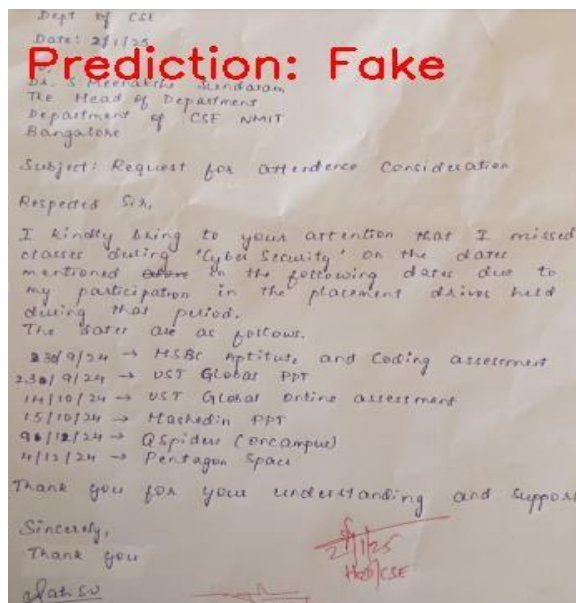


Fig. 20 Image Editing Classification: Fake

Fig. 20 shows the image editing detection model's results, the input image and the predicted output as fake.

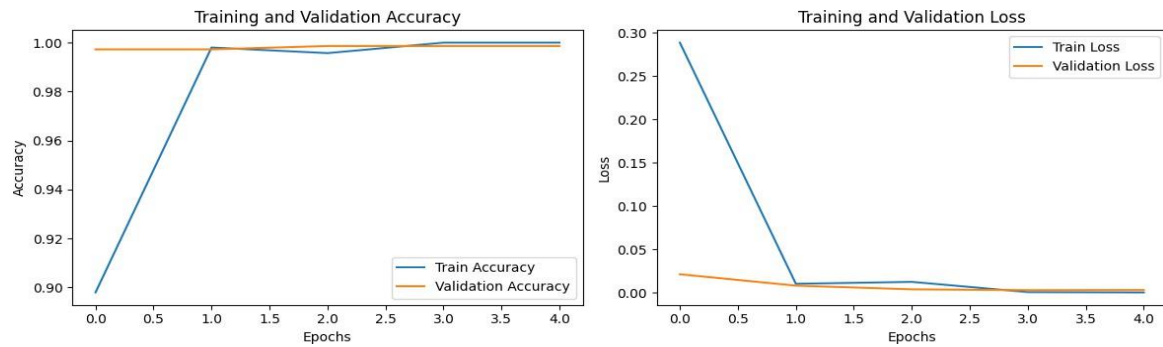


Fig. 21 Training and Validation Accuracy and Loss graph for CNN Face Recognition

Fig. 21 shows two plots: Training and Validation Accuracy, and Training and Validation Loss. The accuracy plot shows the training and validation accuracy increasing over the epochs, while the loss plot shows the training and validation loss decreasing over the epochs.

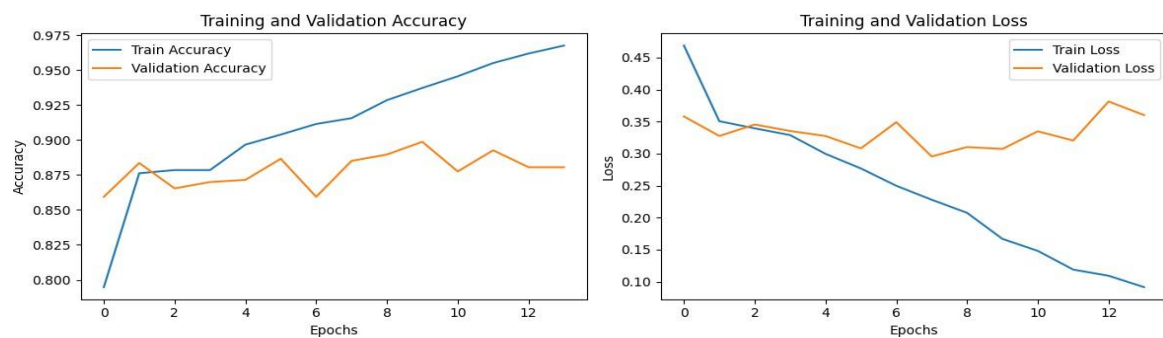


Fig. 22 Training and Validation Accuracy and Loss graph for CNN Image Editing Classification

Fig. 22 shows two plots: Training and Validation Accuracy, and Training and Validation Loss. The accuracy plot shows the training and validation accuracy increasing over the epochs, with the training accuracy slightly higher than the validation accuracy. The loss plot shows the training and validation loss decreasing over the epochs, with the validation loss being slightly higher than the training loss.

Algorithm	Accuracy	Precision	Recall	F1-Score	Test Loss
CNN	99.37%	-	-	-	0.0222
SVM	100.00%	1.00	1.00	1.00	-
KNN	100.00%	1.00	1.00	1.00	-

Table 4. Comparative analysis of Face Recognition Models

- SVM (Support Vector Machine) and KNN (K-Nearest Neighbors) both achieved perfect accuracy (100%), with no misclassifications or confusion, indicating exceptional performance. The confusion matrices show no cross-class errors, suggesting that both algorithms were able to identify each individual perfectly.
- The CNN (Convolutional Neural Network) also performs exceptionally well with an accuracy of 99.37%, which is very close to perfect. However, the slight difference in accuracy compared to SVM and KNN suggests that CNN could be less sensitive to certain nuances in the data or might require further tuning or more data.

Algorithm	Accuracy	Precision	Recall	F1-Score	Test Loss
CNN	88.81%	-	-	-	0.3168
CNN+LSTM	89.12%	-	-	-	0.2385
Random Forest	77.25%	0.77	0.77	0.77	-

Table 5. Comparative analysis of Image Editing Classification Models

- CNN and CNN+LSTM performed similarly with high accuracy, but CNN+LSTM slightly edged out CNN with an accuracy of 89.12%. The combination of CNN and LSTM is likely benefitting from LSTM's ability to process sequential or temporal features, which might be relevant in image editing tasks where patterns evolve across time or sequence.
- Random Forest performed considerably lower, with an accuracy of 77.25%, indicating that it is less suited for this task compared to deep learning models like CNN and CNN+LSTM. The precision, recall, and F1-scores are also lower, showing that the model struggled with classifying the images correctly.

CHAPTER 6 CONCLUSION AND FUTURE WORK

This study demonstrated the successful application of AI and machine learning in face recognition, image editing classification, and natural language processing, yielding impressive results. Notably, the CNN model achieved an outstanding 99.37% accuracy in face recognition, while the hybrid CNN+LSTM model excelled with an 89.12% accuracy in image editing classification. These findings highlight the growing effectiveness of deep learning techniques in addressing complex tasks, with potential applications in fields like security, entertainment, and digital forensics. Despite these successes, there are limitations, such as challenges in recognizing faces from different angles and detecting various types of image editing beyond spliced images. The significance of this study lies in showcasing the potential of AI to solve real-world problems, and future research should focus on refining model architectures and hyperparameters, as well as utilizing more diverse datasets to enhance model performance and adaptability across a wider range of scenarios. Future research should concentrate on optimizing model architectures and hyperparameters to further enhance performance and using diverse datasets to enhance their performance in different real-world scenarios.

Bibliography

- [1]J. Cheng, "Design of Intelligent Autonomous Learning Management System Based on Artificial Intelligence Technology," 2022 International Conference on Education, Network and Information Technology (ICENIT), Liverpool, United Kingdom, 2022, pp. 106-110, doi: 10.1109/ICENIT57306.2022.00030..
- [2]S. Dutta, S. Ranjan, S. Mishra, V. Sharma, P. Hewage and C. Iwendi, "Enhancing Educational Adaptability: A Review and Analysis of AI Driven Adaptive Learning Platforms," 2024 4th International Conference on Innovative Practices in Technology and Management (ICIPTM), Noida, India, 2024, pp. 1-5, doi: 10.1109/ICIPTM59628.2024.10563448.
- [3]S. K. Vishnumolakala, S. C. C, N. P. Subheesh, P. Kumar and R. Kumar, "AI-Based Research Companion (ARC): An Innovative Tool for Fostering Research Activities in Undergraduate Engineering Education," 2024 IEEE Global Engineering Education Conference (EDUCON), Kos Island, Greece, 2024, pp. 1-5, doi: 10.1109/EDUCON60312.2024.10578646.
- [4]S. Zharmukhanbetov and C. P. Singh, "Enhancing Flipped Classroom Engagement and Personalized Learning Through AI-Powered Adaptive Content Delivery," 2023 3rd International Conference on Technological Advancements in Computational Sciences (ICTACS), Tashkent, Uzbekistan, 2023, pp. 1411-1416, doi: 10.1109/ICTACS59847.2023.10389841.
- [5]Y. Du, W. Song, Y. Coa, X. Chen and G. Huang, "Design of Personalized AI Examination System Based on Reinforcement Learning," 2024 6th International Conference on Computer Science and Technologies in Education (CSTE), Xi'an, China, 2024, pp. 119-124, doi: 10.1109/CSTE62025.2024.00029.
- [6]B. Pardamean, T. Suparyanto, T. W. Cenggoro, D. Sudigyo, A. Anugrahana and I. Anugraheni, "Model of Learning Management System Based on Artificial Intelligence in Team- Based Learning Framework," 2021 International Conference on Information

Management and Technology (ICIMTech), Jakarta, Indonesia, 2021, pp. 37-42, doi: 10.1109/ICIMTech53080.2021.9535088.

[7]. Xiao and Q. Bai, "iTutor: Promoting AI Guided Knowledge Interaction in Online Learning," 2022 International Symposium on Educational Technology (ISET), Hong Kong, Hong Kong, 2022, pp. 253-257, doi: 10.1109/ISET55194.2022.00061.

[8]J. Harikrishnan, A. Sudarsan, A. Sadashiv and R. A. S. Ajai, "Vision Face Recognition Attendance Monitoring System for Surveillance using Deep Learning Technology and Computer Vision," 2019 International Conference on Vision Towards Emerging Trends in Communication and Networking (ViTECoN), Vellore, India, 2019, pp. 1-5, doi: 10.1109/ViTECoN.2019.8899418.

[9]H. Kesharwani, T. Mallick, A. Gupta and G. Raj, "Automated Attendance System Using Computer Vision," 2022 Second International Conference on Computer Science, Engineering and Applications (ICCSEA), Gunupur, India, 2022, pp. 1-5, doi: 10.1109/ICCSEA54677.2022.9936266.

[10]G. A. Senthil, S. Geerthik, R. Karthikeyan and G. Keerthana, "Face Recognition based Automated Smart Attendance using Hybrid Machine Learning Algorithms and Computer Vision," 2024 3rd International Conference on Applied Artificial Intelligence and Computing (ICAAIC), Salem, India, 2024, pp. 606-611, doi: 10.1109/ICAAIC60222.2024.10574896.

[11]R. Ai, D. Kohli, L. Maina, E. Manelski, N. T. Dhar and C. Brzycki, "Programming an Educational Chatbot to Support Virtual Learning," 2020 IEEE MIT Undergraduate Research Technology Conference (URTC), Cambridge, MA, USA, 2020, pp. 1-5, doi: 10.1109/URTC51696.2020.9668855.

[12]V. Bisht, R. Choyal, A. S. Negi and E. K. Singh, "Utilizing Python for Web Scraping and Incremental Data Extraction," 2023 2nd International Conference on Automation, Computing and Renewable Systems (ICACRS), Pudukkottai, India, 2023, pp. 1450-1455, doi: 10.1109/ICACRS58579.2023.10404702.

- [13]P. Isaac Ritharson, D. Sujitha Juliet, J. Anitha and S. Immanuel Alex Pandian, "Multi- Document Summarization Made Easy: An Abstractive Query-Focused System Using Web Scraping and Transformer Models," 2023 3rd International Conference on Intelligent Technologies (CONIT), Hubli, India, 2023, pp. 1-6, doi: 10.1109/CONIT59222.2023.10205946.
- [14]A. A. Sukmandhani, T. Sunjaya, I. P. Saputro and J. Ohliati, "Data Scraping using Python for Information Retrieval on E-Commerce with Brand Keyword," 2023 8th International Conference on Business and Industrial Research (ICBIR), Bangkok, Thailand, 2023, pp. 179- 183, doi: 10.1109/ICBIR57571.2023.10147717.
- [15]V. Suresh, R. Agasthiya, J. Ajay, A. A. Gold and D. Chandru, "AI based Automated Essay Grading System using NLP," 2023 7th International Conference on Intelligent Computing and Control Systems (ICICCS), Madurai, India, 2023, pp. 547-552, doi: 10.1109/ICICCS56967.2023.10142822.
- [16]J. S. Kusuma, K. Halim, E. J. P. Pranoto, B. Kanigoro and E. Irwansyah, "Automated Essay Scoring Using Machine Learning," 2022 4th International Conference on Cybernetics and Intelligent System (ICORIS), Prapat, Indonesia, 2022, pp. 1-5, doi: 10.1109/ICORIS56080.2022.10031338.
- [17]Z. Chen and Y. Zhou, "Research on Automatic Essay Scoring of Composition Based on CNN and OR," 2019 2nd International Conference on Artificial Intelligence and Big Data (ICAIBD), Chengdu, China, 2019, pp. 13-18, doi: 10.1109/ICAIBD.2019.8837007.
- [18]S. E. VP, C. M. S and R. Dheepthi, "LLM-Enhanced Deepfake Detection: Dense CNN and Multi-Modal Fusion Framework for Precise Multimedia Authentication," 2024 International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS), Chennai, India, 2024, pp. 1-6, doi: 10.1109/ADICS58448.2024.10533511.

[19]D. Huang and Y. Zhang, "Learning Meta Model for Strong Generalization Deepfake Detection," 2024 International Joint Conference on Neural Networks (IJCNN), Yokohama, Japan, 2024, pp. 1-8, doi: 10.1109/IJCNN60899.2024.10651482

[20]N. Kumaladewi, G. N. Refardi, M. N. Gunawan and Zulkifli, "Design and Build a Web- Based e-Learning System using ReactJS Framework," 2024 International Conference on Advances in Computer Science and Engineering, 2024, doi: 10.1109/ICACSE.2024.10455534

Appendix – A

Appendix – B

Publication Details